TD7: Ingeniería de Datos

Universidad Torcuato Di Tella

Resumen de Clases Teóricas

Teóricas 1.0 a 7.4

Abril 2026

Índice

1. T1.0 — Presentación del curso **4**

2. T1.1 — Introducción a las bases de datos **4**
2.1. ¿Qué es una base de datos? 4
2.2. ¿Por qué no usar archivos planos? 4
2.3. Niveles de abstracción 4
2.4. Lenguajes 4
2.5. Propiedades ACID 5

3. T1.2 — Primary Key y Foreign Key **5**
3.1. Comportamiento ante DELETE/UPDATE en la tabla padre 5

4. T2 — Modelo Entidad-Interrelación (DER) **5**
4.1. Componentes 5
4.2. Tipos de atributos 5
4.3. Cardinalidad 6
4.4. Participación 6
4.5. Entidades débiles 6
4.6. Especialización/Generalización (herencia) 6
4.7. Paso al modelo relacional 6

5. T3.1 — Modelo Relacional **6**
5.1. Conceptos clave 7
5.2. Propiedades de una relación 7
5.3. Restricciones de integridad 7
5.4. Jerarquía de claves 7
5.5. SQL DDL 7

6. T3.2 — Álgebra Relacional **8**
6.1. Operadores unarios 8
6.2. Operadores de conjuntos 8
6.3. Producto cartesiano y junta 8

7. T3.3 — Introducción a SQL **8**
7.1. Estructura básica 8
7.2. Funciones de agregación 9
7.3. JOIN 9

8. T4.1 — SQL Parte II **9**
8.1. ORDER BY 9
8.2. Paginación 9
8.3. GROUP BY 9
8.4. HAVING 10
8.5. Orden de ejecución lógico 10
8.6. OUTER JOIN 10

9. T4.2 — SQL Parte III **10**

9.1. Cláusula WITH (CTE) . . . 10
9.2. Subqueries (consultas anidadas) . . . 10
9.3. Operaciones de conjuntos . . . 11
9.4. Diferencias entre COUNT . . . 11

10. T5.1 — Funciones de Ventana (Window Functions) 11
10.1. Sintaxis general . . . 11
10.2. Funciones de ranking . . . 12
10.3. Funciones de desplazamiento . . . 12
10.4. Ejemplo . . . 12

11. T5.2 — Normalización 12
11.1. Anomalías de la redundancia . . . 13
11.2. Dependencias funcionales . . . 13
11.3. Formas normales . . . 13
11.3.1. 1FN — Primera Forma Normal . . . 13
11.3.2. 2FN — Segunda Forma Normal . . . 13
11.3.3. 3FN — Tercera Forma Normal . . . 13
11.3.4. FNBC — Forma Normal de Boyce-Codd . . . 13
11.4. Descomposición . . . 13

12. T6.1 — Concurrencia: Serializabilidad 14
12.1. Motivación . . . 14
12.2. Schedule y serializabilidad . . . 14
12.3. Conflictos . . . 14
12.4. Grafo de precedencias . . . 14
12.5. Control de concurrencia . . . 14
12.5.1. Estrategia optimista (timestamps) . . . 14
12.5.2. Estrategia pesimista (locks) . . . 15
12.5.3. Protocolo 2PL (Two-Phase Locking) . . . 15
12.5.4. Peligros del 2PL . . . 15

13. T6.2 — Concurrencia: Recuperabilidad 15
13.1. Motivación . . . 15
13.2. Schedule recuperable . . . 15
13.3. Rollback en cascada . . . 15
13.4. Log de operaciones . . . 16
13.5. Resumen: independencia entre propiedades . . . 16

14. T6.3 — Recuperación ante Fallas 16
14.1. Tipos de fallas . . . 16
14.2. El WAL (Write-Ahead Log) . . . 16
14.3. Reglas del Gestor de Recuperación . . . 16
14.4. Estrategias de recuperación . . . 17
14.4.1. UNDO (actualización inmediata) . . . 17
14.4.2. REDO (actualización diferida) . . . 17
14.4.3. Checkpoints en REDO . . . 17

15. T7.1 — Recuperación: UNDO/REDO 17

15.1. Estrategia UNDO/REDO (actualización libre) . . . 17
15.2. Procedimiento de recuperación bajo UNDO/REDO . . . 17
15.3. Comparación de estrategias . . . 18

16. T7.2 — Backups **18**
16.1. Necesidad . . . 18
16.2. Dumps periódicos . . . 18
16.3. Tipos de dump . . . 18
16.4. Analogía checkpoint/dump . . . 18
16.5. High Availability — Réplicas . . . 18
16.6. RAID . . . 19

17. T7.3 — Almacenamiento e Índices **19**
17.1. Jerarquía de almacenamiento . . . 19
17.2. Discos magnéticos . . . 19
17.2.1. Técnicas de optimización . . . 19
17.3. Organización de archivos — Registros . . . 20
17.3.1. Registros de tamaño fijo . . . 20
17.3.2. Registros de tamaño variable . . . 20
17.4. Metadata (Data Dictionary) . . . 20
17.5. Índices . . . 20
17.5.1. Motivación . . . 20
17.5.2. Clustered vs. Unclustered . . . 20
17.5.3. Tipos de índices . . . 20
17.5.4. Sintaxis SQL . . . 21

18. T7.4 — Procesamiento de Consultas **21**
18.1. Pipeline completo . . . 21
18.2. Información de catálogo (estadísticas) . . . 21
18.3. Estimación de costo . . . 21
18.3.1. Costos de selección (σ) . . . 22
18.4. Selecciones complejas . . . 22
18.5. Plan de ejecución . . . 22

Mapa conceptual del curso **22**

T1.0 — Presentación del curso

Presentación general de la materia. El curso cubre el ciclo completo de trabajo con datos: desde el modelado y diseño de bases de datos relacionales, pasando por SQL, hasta temas avanzados de gestión (concurrencia, recuperación, almacenamiento). Se usa **PostgreSQL** como motor y la base de datos **Chinook** como dataset de práctica.

T1.1 — Introducción a las bases de datos

¿Qué es una base de datos?

Colección organizada de datos interrelacionados, gestionada por un **SGBD** (Sistema Gestor de Bases de Datos).

¿Por qué no usar archivos planos?

Los archivos tienen problemas de redundancia, inconsistencia, dificultad de acceso, falta de integridad, concurrencia imposible y ausencia de seguridad. Los SGBD resuelven todo esto.

Niveles de abstracción

- **Físico:** cómo se almacenan los datos en disco
- **Lógico (conceptual):** qué datos hay y sus relaciones
- **Vistas:** subconjuntos de datos para distintos usuarios

Independencia de datos: cambios en el nivel físico no afectan al lógico (independencia física), y cambios en el lógico no afectan a las aplicaciones (independencia lógica).

Lenguajes

- **DDL** (Data Definition Language): define esquemas
- **DML** (Data Manipulation Language): consulta y modifica datos
- **DCL** (Data Control Language): permisos
- **TCL** (Transaction Control Language): transacciones

Propiedades ACID

- **Atomicity:** la transacción se ejecuta completa o no se ejecuta
- **Consistency:** lleva la base de un estado consistente a otro
- **Isolation:** las transacciones concurrentes se comportan como si fueran seriales
- **Durability:** los cambios confirmados son permanentes

T1.2 — Primary Key y Foreign Key

Clave Primaria (PK): atributo o conjunto de atributos que identifican unívocamente cada fila de una tabla. No puede ser NULL ni repetirse. Se elige entre las *claves candidatas*.

Clave Foránea (FK): atributo(s) en una tabla que referencia(n) la PK de otra tabla. Garantiza la **integridad referencial**: no puede existir un valor en la FK que no exista en la tabla referenciada.

Comportamiento ante DELETE/UPDATE en la tabla padre

- `CASCADE`: se propaga el cambio
- `SET NULL`: la FK se pone en NULL
- `RESTRICT` / `NO ACTION`: se rechaza la operación

T2 — Modelo Entidad-Interrelación (DER)

Herramienta de diseño conceptual para modelar la realidad *antes* de implementar.

Componentes

- **Entidades:** objetos del mundo real (rectángulos). Tienen **atributos** (óvalos). El atributo identificador es la *clave* (subrayado).
- **Interrelaciones:** vínculos entre entidades (rombos). También pueden tener atributos propios.

Tipos de atributos

- Simples vs. compuestos
- Univaluados vs. multivaluados (doble óvalo)

- Derivados (línea punteada)
- Clave (subrayado)

Cardinalidad

- **1:1** — uno a uno
- **1:N** — uno a muchos
- **N:M** — muchos a muchos

Participación

- **Total** (línea doble): toda entidad participa obligatoriamente
- **Parcial** (línea simple): la participación es opcional

Entidades débiles

No tienen clave propia; dependen de una *entidad identificadora*. Se representan con rectángulo doble. Su clave es una *clave parcial* (discriminante).

Especialización/Generalización (herencia)

Una entidad puede especializarse en subentidades (**ISA**). Puede ser disjunta o solapada, total o parcial.

Paso al modelo relacional

- Cada entidad → tabla
- Relaciones N:M → tabla intermedia
- Relaciones 1:N → FK en el lado "N"
- Entidades débiles → incluyen la PK de la entidad identificadora

T3.1 — Modelo Relacional

Propuesto por **Edgar Codd** en 1970. La base de datos es un conjunto de **relaciones** (tablas).

Conceptos clave

- **Relación:** subconjunto del producto cartesiano de dominios
- **Esquema:** $R(A_1, A_2, \ldots, A_n)$
- **Tupla:** fila de la tabla
- **Dominio:** conjunto de valores válidos para un atributo

Propiedades de una relación

- No hay tuplas duplicadas
- Los atributos no tienen orden
- Los valores de los atributos son atómicos (1FN)

Restricciones de integridad

1. **De dominio:** el valor debe pertenecer al dominio del atributo
2. **De unicidad:** la PK identifica cada tupla unívocamente
3. **Integridad de entidad:** la PK no puede ser NULL
4. **Integridad referencial:** las FK deben apuntar a valores existentes

Jerarquía de claves

- **Superclave:** cualquier conjunto de atributos que identifica unívocamente
- **Clave candidata:** superclave mínima (sin atributos redundantes)
- **Clave primaria:** clave candidata elegida como identificador oficial

SQL DDL

```sql
CREATE TABLE empleado (
    legajo     INT           PRIMARY KEY,
    nombre     VARCHAR(50)   NOT NULL,
    salario    NUMERIC(10,2),
    dept_id    INT           REFERENCES departamento(id)
                             ON DELETE SET NULL
);
```

Tipos de datos: `INT`, `FLOAT`, `NUMERIC(i,j)`, `VARCHAR(n)`, `DATE`, `TIME(i)`, `BOOLEAN`.

Restricciones: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`.

T3.2 — Álgebra Relacional

Lenguaje **formal y procedural** que describe *cómo* obtener el resultado. Base teórica de SQL.

Operadores unarios

Operador	Notación	Descripción
Selección	$\sigma_{\text{cond}}(R)$	Filtra filas que cumplen la condición
Proyección	$\pi_{\text{atrs}}(R)$	Selecciona columnas (elimina duplicados)
Renombramiento	$\rho_{\text{nuevo}}(R)$	Renombra la relación o atributos

Operadores de conjuntos

Requieren relaciones *compatibles* (mismos atributos y tipos).

Operador	Símbolo	Descripción
Unión	$R \cup S$	Tuplas en R o en S (sin duplicados)
Diferencia	$R - S$	Tuplas en R pero no en S
Intersección	$R \cap S$	Tuplas en R y en S

Producto cartesiano y junta

- **Producto cartesiano:** $R \times S$ — combina todas las tuplas de R con todas las de S
- **Junta (Join):** $R \bowtie_{\text{cond}} S = \sigma_{\text{cond}}(R \times S)$
- **Junta natural:** $R \bowtie S$ — une por atributos con el mismo nombre

T3.3 — Introducción a SQL

SQL es un lenguaje **declarativo** (especifica *qué*, no *cómo*).

Estructura básica

```
SELECT atributos
FROM   tabla
WHERE  condicion;
```

Dataset de práctica — Chinook: tienda de media digital. 11 tablas: Artist, Album, Track, Genre, MediaType, Invoice, InvoiceLine, Customer, Employee, Playlist, PlaylistTrack. Más de 15.000 filas.

Funciones de agregación

- `COUNT(*)` — cuenta filas (incluye NULLs)
- `COUNT(A)` — cuenta valores no NULL del atributo A
- `COUNT(DISTINCT A)` — cuenta valores distintos no NULL
- `SUM(A)`, `AVG(A)`, `MIN(A)`, `MAX(A)`

JOIN

```
SELECT a.Title, ar.Name
FROM   Album a
INNER JOIN Artist ar ON a.ArtistId = ar.ArtistId;
```

Operaciones con NULL: cualquier comparación con NULL da UNKNOWN. Usar `IS NULL` / `IS NOT NULL`.

T4.1 — SQL Parte II

ORDER BY

```
SELECT * FROM Track ORDER BY UnitPrice DESC, Name ASC;
```

Paginación

```
SELECT * FROM Track
ORDER BY TrackId
OFFSET 10 ROWS FETCH FIRST 5 ROWS ONLY;
```

GROUP BY

```
SELECT GenreId, COUNT(*) AS cantidad
FROM   Track
GROUP BY GenreId;
```

Todo atributo en SELECT que no sea función de agregación **debe estar** en GROUP BY.

HAVING

Filtra grupos (se aplica *después* de GROUP BY, a diferencia de WHERE que filtra filas):

```
SELECT GenreId, COUNT(*) AS cantidad
FROM   Track
GROUP BY GenreId
HAVING COUNT(*) > 50;
```

Orden de ejecución lógico

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

OUTER JOIN

```
-- LEFT JOIN: todas las filas de la izquierda, NULL si no hay
   match
SELECT c.CustomerId, i.InvoiceId
FROM   Customer c
LEFT JOIN Invoice i ON c.CustomerId = i.CustomerId;

-- RIGHT JOIN: todas las filas de la derecha
-- FULL OUTER JOIN: todas las filas de ambas tablas
```

T4.2 — SQL Parte III

Cláusula WITH (CTE)

```
WITH ventas_por_cliente AS (
    SELECT CustomerId, SUM(Total) AS total
    FROM   Invoice
    GROUP BY CustomerId
)
SELECT * FROM ventas_por_cliente WHERE total > 100;
```

Permite nombrar subconsultas y reutilizarlas. Mejora la legibilidad y permite encadenar resultados intermedios.

Subqueries (consultas anidadas)

```
-- Escalar
SELECT * FROM Track
WHERE Milliseconds > (SELECT AVG(Milliseconds) FROM Track);
```

```sql
-- Con IN
SELECT * FROM Customer
WHERE CustomerId IN (SELECT CustomerId FROM Invoice);

-- Con ALL / SOME
SELECT * FROM Track
WHERE UnitPrice > ALL (SELECT UnitPrice FROM Track WHERE
   GenreId = 1);

-- Con EXISTS (correlacionada)
SELECT * FROM Artist a
WHERE EXISTS (
    SELECT 1 FROM Album al WHERE al.ArtistId = a.ArtistId
);
```

Operaciones de conjuntos

```sql
-- UNION (elimina duplicados)
SELECT City FROM Customer UNION SELECT City FROM Employee;

-- UNION ALL (mantiene duplicados)
SELECT City FROM Customer UNION ALL SELECT City FROM Employee;

-- INTERSECT
SELECT City FROM Customer INTERSECT SELECT City FROM Employee;

-- EXCEPT (diferencia)
SELECT City FROM Customer EXCEPT SELECT City FROM Employee;
```

Diferencias entre COUNT

- `COUNT(*)` — cuenta todas las filas, incluyendo NULLs
- `COUNT(A)` — cuenta solo valores no NULL de A
- `COUNT(DISTINCT A)` — cuenta valores distintos no NULL de A

T5.1 — Funciones de Ventana (Window Functions)

Las funciones de ventana calculan valores sobre un subconjunto de filas relacionadas con la fila actual, **sin colapsar el resultado** (a diferencia de GROUP BY).

Sintaxis general

```
funcion() OVER (
    [PARTITION BY atributo]
    [ORDER BY atributo]
    [ROWS/RANGE frame_specification]
)
```

Funciones de ranking

- `ROW_NUMBER()` — número de fila único (sin empates)
- `RANK()` — ranking con saltos ante empates (1, 1, 3...)
- `DENSE_RANK()` — ranking sin saltos (1, 1, 2...)
- `NTILE(n)` — divide en n grupos iguales

Funciones de desplazamiento

- `LAG(expr, n)` — valor de n filas antes
- `LEAD(expr, n)` — valor de n filas adelante
- `FIRST_VALUE(expr)` — primer valor de la ventana
- `LAST_VALUE(expr)` — último valor de la ventana

Ejemplo

```
SELECT nombre, salario,
       AVG(salario) OVER (PARTITION BY departamento) AS
          avg_dept,
       SUM(salario) OVER (ORDER BY fecha
                          ROWS UNBOUNDED PRECEDING)  AS
                             acumulado
FROM empleados;
```

PARTITION BY: divide las filas en grupos (como GROUP BY pero sin colapsar). Cada partición define una ventana independiente.

T5.2 — Normalización

La normalización reduce **redundancia** y evita **anomalías**.

Anomalías de la redundancia

- **Inserción:** para insertar un dato hay que repetir otros
- **Modificación:** hay que actualizar en múltiples lugares
- **Eliminación:** al borrar se pierde información no deseada

Dependencias funcionales

$X \to Y$ significa que conocer X determina unívocamente el valor de Y.

- **Dependencia completa:** Y depende de todo X (no de un subconjunto)
- **Dependencia transitiva:** $X \to Y \to Z$ (donde Y no es clave)

Formas normales

1FN — Primera Forma Normal

Todos los atributos son atómicos (sin grupos repetidos ni atributos multivaluados). Es la definición misma del modelo relacional.

2FN — Segunda Forma Normal

Está en 1FN y **no hay dependencias parciales**: todo atributo no primo depende de la clave completa. Solo es relevante cuando la PK es compuesta.

3FN — Tercera Forma Normal

Está en 2FN y **no hay dependencias transitivas**: ningún atributo no primo depende de otro atributo no primo.

FNBC — Forma Normal de Boyce-Codd

Versión más estricta de 3FN. Para toda dependencia funcional $X \to Y$ no trivial, X debe ser superclave.

Descomposición

Para normalizar se divide una relación problemática. La descomposición debe ser:

- **Sin pérdida (lossless):** el join de las partes reconstruye la relación original
- **Que preserve dependencias:** las DF originales siguen siendo verificables

T6.1 — Concurrencia: Serializabilidad

Motivación

En producción, múltiples transacciones se ejecutan simultáneamente. Sin control, pueden surgir inconsistencias.

Schedule y serializabilidad

- **Schedule:** orden intercalado de instrucciones de múltiples transacciones
- **Schedule serial:** las transacciones se ejecutan de a una, sin intercalarse. Siempre correcto.
- **Schedule serializable:** produce el mismo resultado que algún schedule serial. Garantiza la propiedad *Isolation* de ACID.

Conflictos

Dos instrucciones de distintas transacciones sobre el mismo ítem conflictúan si al menos una es escritura:

- $R_{T1}(X),\ W_{T2}(X)$ — T1 lee lo que T2 va a escribir
- $W_{T1}(X),\ R_{T2}(X)$ — T1 escribe lo que T2 va a leer
- $W_{T1}(X),\ W_{T2}(X)$ — ambas escriben el mismo ítem

Un schedule es **serializable por conflictos** si puede transformarse en un schedule serial mediante inversiones de instrucciones no conflictivas.

Grafo de precedencias

- Nodos = transacciones
- Arco $T_1 \to T_2$ si hay un conflicto de una instrucción de T_1 que precede a una de T_2
- **El schedule es serializable $\iff$ el grafo no tiene ciclos**

Control de concurrencia

Estrategia optimista (timestamps)

Asume que los conflictos son raros. Deja operar libremente y resuelve conflictos con rollback.

Estrategia pesimista (locks)

Evita conflictos bloqueando recursos antes de operar.

- **Lock exclusivo (X):** para escritura. Ningún otro lock puede concederse sobre ese recurso.
- **Lock compartido (S):** para lectura. Pueden coexistir múltiples S-locks; no puede coexistir con X-lock.

Protocolo 2PL (Two-Phase Locking)

Garantiza serializabilidad. Regla: una transacción **no puede adquirir ningún lock después de haber liberado alguno**.

- **Fase de crecimiento:** adquiere locks
- **Fase de reducción:** libera locks

Peligros del 2PL

- **Deadlock:** dos transacciones se esperan mutuamente
- **Starvation:** una transacción nunca consigue el lock porque otras siempre se le adelantan

T6.2 — Concurrencia: Recuperabilidad

Motivación

La serializabilidad garantiza *Isolation*. También se necesita *Atomicity* y *Durability* cuando una transacción falla dentro de un schedule complejo.

Schedule recuperable

Un schedule es **recuperable** si y solo si ninguna transacción T hace commit hasta que todas las transacciones que escribieron datos que T leyó hayan hecho commit primero.

Formalmente: si $W_{T1}(X)$ ocurre antes que $R_{T2}(X)$, entonces c_{T1} debe ocurrir antes que c_{T2}.

Rollback en cascada

Si T1 aborta y T2 leyó datos de T1, hay que hacer rollback de T2 también, y de todo lo que dependa de T2.

Log de operaciones

Para poder deshacer efectos, el motor mantiene un log con registros:

- `BEGIN` T_{id}
- `WRITE` T_{id}, `X`, X_{old}, X_{new}
- `READ` T_{id}, `X`
- `COMMIT` T_{id}
- `ABORT` T_{id}

Resumen: independencia entre propiedades

Serializabilidad y recuperabilidad son propiedades **independientes**. Un schedule puede ser serializable sin ser recuperable, y viceversa.

T6.3 — Recuperación ante Fallas

Tipos de fallas

1. **Fallas de sistema:** errores de software/hardware que detienen el gestor (segfault, división por cero). No hay pérdida de información en disco.
2. **Fallas de dispositivos:** daño físico en disco o memoria. Puede haber pérdida de datos.
3. **Fallas naturales externas:** cortes de luz, terremotos, incendios.

Fallas catastróficas (pérdida de datos en disco) → requieren **backups**.
Fallas no catastróficas (solo se interrumpe la ejecución) → requieren **mecanismos de recuperación** basados en log.

El WAL (Write-Ahead Log)

Archivo que registra *todas* las operaciones del gestor. Registros: (`BEGIN`, T_{id}), (`WRITE`, T_{id}, `X`, v_{old}, v_{new}), (`READ`, T_{id}, `X`), (`COMMIT`, T_{id}), (`ABORT`, T_{id}).

Reglas del Gestor de Recuperación

1. **Regla WAL:** antes de escribir un ítem modificado en disco, el registro de log correspondiente debe estar ya en disco.
2. **Regla FLC (Force Log at Commit):** cuando una transacción hace commit, el log debe volcarse a disco (*flush*).

Estrategias de recuperación

UNDO (actualización inmediata)

Los datos se escriben en disco lo antes posible, *antes* del commit. El log guarda v_{old}. Ante una falla: recorrer el log de atrás hacia adelante y restaurar v_{old} para transacciones sin COMMIT. El algoritmo es **idempotente**: puede ejecutarse parcialmente múltiples veces con el mismo resultado final.

REDO (actualización diferida)

Los datos **no** se escriben en disco hasta después del commit. El log guarda v_{new}. Ante una falla: buscar transacciones commiteadas y rehacer sus escrituras. Requiere **checkpoints** para no tener que retroceder hasta el inicio del log.

Checkpoints en REDO

1. Escribir `(BEGIN CKPT, transacciones_activas)` en el log
2. Volcar a disco todos los ítems modificados por transacciones ya commiteadas
3. Escribir `(END CKPT)` en el log y volcar el log a disco

T7.1 — Recuperación: UNDO/REDO

Estrategia UNDO/REDO (actualización libre)

El SGBD puede volcar ítems a disco cuando quiera (antes o después del commit). Mayor flexibilidad de gestión de memoria.

El log guarda tanto v_{old} como v_{new}: `(WRITE,` T_i`, X,` v_{old}`,` v_{new}`)`.

Regla maestra: el log se vuelca a disco *antes* de escribir el ítem (Write-Ahead Logging puro).

Procedimiento de recuperación bajo UNDO/REDO

1. Recorrer el log de atrás hacia adelante, clasificando transacciones en commiteadas y no commiteadas
2. Para las no commiteadas: deshacer escrituras (restaurar v_{old})
3. En una pasada de adelante hacia atrás: rehacer escrituras de transacciones commiteadas
4. Registrar `ABORT` para las no commiteadas y volcar el log

Comparación de estrategias

Estrategia	Escribe antes del commit	Rehace	Deshace
UNDO	Sí (obligatorio)	No	Sí
REDO	No	Sí	No
UNDO/REDO	Libre	Sí	Sí

T7.2 — Backups

Necesidad

Ante fallas catastróficas (daño físico), el log no alcanza. Se necesitan copias externas de la base.

Dumps periódicos

Copias realizadas periódicamente (ej. una vez al día) a un soporte externo, idealmente en otra ubicación física.

- Al recuperar: restaurar el dump y reproducir el log para rehacer las transacciones ocurridas desde el dump.
- Si el log también fue dañado: se pierden las transacciones más recientes.

Tipos de dump

- **Full dump:** copia completa de toda la base de datos
- **Incremental dump:** solo los datos modificados desde el último dump

Analogía checkpoint/dump

- **Checkpoint:** mueve datos de memoria a disco → recuperación de fallas de sistema
- **Dump:** mueve datos de disco a archivo externo → recuperación de fallas de dispositivos

High Availability — Réplicas

- Para fallas de gran escala se usan **sistemas de réplicas** en distintas ubicaciones
- El backup remoto se denomina **réplica secundaria**

- Arquitectura **Activo/Pasivo**: el nodo secundario solo sirve tráfico cuando el primario está caído
- La replicación continua se realiza a través del log (el primario envía entradas al secundario)

RAID

Técnica de gestión de discos para mejorar rendimiento y/o tolerancia a fallos:

- **RAID 0 (Striping):** distribuye datos entre discos para mayor velocidad; sin redundancia
- **RAID 1 (Mirroring):** duplica los datos en dos discos
- **RAID 5 (Paridad):** combina rendimiento y tolerancia a fallos con paridad distribuida

T7.3 — Almacenamiento e Índices

Jerarquía de almacenamiento

- **Primario:** registros del procesador, caché, RAM — muy rápido, volátil, costoso
- **Secundario:** flash/SSD, discos magnéticos — lento, persistente, barato
- **Terciario:** cintas magnéticas — muy lento, gran capacidad, largo plazo

Discos magnéticos

La unidad de I/O es el **bloque** (4 KB–16 KB). Se leen/escriben de forma atómica. El acceso puede ser:

- **Secuencial:** bloques físicamente adyacentes → más rápido
- **Aleatorio (random access):** bloques dispersos → más lento (requiere seek)

Técnicas de optimización

- **Buffering:** mantener bloques ya leídos en memoria para evitar releerlos
- **Read-ahead:** cargar bloques consecutivos anticipadamente
- **Scheduling:** reordenar pedidos de bloques según su ubicación física

Organización de archivos — Registros

Los registros son el correlato en disco de las tuplas.

Registros de tamaño fijo

- Facilita el almacenamiento pero impide dominios de tamaño variable (como `VARCHAR`)
- Cada bloque aloja $\lfloor$tamaño_bloque/tamaño_registro$\rfloor$ registros
- Al borrar registros quedan "huecos" reutilizables en futuras inserciones

Registros de tamaño variable

- Permite dominios de distinto tamaño y múltiples relaciones en el mismo archivo
- Cada bloque tiene un header con: cantidad de registros, lista de ubicaciones de cada registro, ubicación del espacio vacío

Metadata (Data Dictionary)

El motor mantiene: nombres de relaciones y atributos con sus tipos, constraints (PK, FK), índices, usuarios y permisos, ubicación física de las relaciones.

Índices

Motivación

Sin índice, una consulta con filtro requiere un **full scan** (leer todos los bloques). Los índices permiten acceder directamente a los registros relevantes.

Clustered vs. Unclustered

- **Clustered (agrupado):** los datos están físicamente ordenados según la clave del índice. Solo puede haber uno por tabla. Muy eficiente para rangos.
- **Unclustered (secundario):** el índice apunta a los registros pero los datos no están físicamente ordenados. Pueden existir varios por tabla.

Tipos de índices

- **B-tree (por defecto):** árbol balanceado. Soporta igualdad y rango (`<`, `<=`, `=`, `>=`, `>`). El más versátil.

- **Hash:** tabla de hash. Solo soporta igualdad (=). Muy rápido para búsquedas exactas, inútil para rangos.
- **GiST, GIN:** para tipos especiales (texto completo, geométricos).

Sintaxis SQL

```
CREATE INDEX idx_nombre
ON    tabla
USING btree
(atributo ASC NULLS LAST)
TABLESPACE pg_default;
```

T7.4 — Procesamiento de Consultas

Pipeline completo

1. **Parser y traductor:** verifica sintaxis/semántica; traduce SQL a álgebra relacional
2. **Optimizer:** elige entre expresiones algebraicas equivalentes; usa estadísticas del catálogo para estimar costos
3. **Execution Engine:** ejecuta el plan elegido y devuelve el resultado

SQL (declarativo: *qué*) → Álgebra relacional (procedural: *cómo*) → Plan de ejecución → Resultado.

Información de catálogo (estadísticas)

- n_r: número de tuplas de la relación r
- b_r: número de bloques con tuplas de r
- l_r: tamaño de una tupla en bytes
- f_r: factor de bloque (tuplas por bloque)
- $V(A, r)$: cantidad de valores distintos del atributo A en r
- $\text{Height}_{I(A,r)}$: altura del árbol B+ del índice I
- $\text{Length}_{I(A,r)}$: tamaño en bloques de las hojas del índice

Estimación de costo

Se mide en función de **seek de bloque** (t_s) y **transferencia de bloque** (t_T).

Costos de selección (σ)

Estrategia	Costo
Full scan	$t_s + b_r \cdot t_T$
Full scan (clave, igualdad)	$t_s + (b_r/2) \cdot t_T$
Clustering index, clave	$(h_i + 1)(t_T + t_s)$
Clustering index, no clave	$h_i(t_T + t_s) + t_s + b \cdot t_T$
Secondary index, clave	$(h_i + 1)(t_T + t_s)$
Secondary index, no clave	$(h_i + n)(t_T + t_s)$
Rango con clustering index	$h_i(t_T + t_s) + t_s + b \cdot t_T$
Rango con secondary index	$(h_i + n)(t_T + t_s)$

Selecciones complejas

- **Conjunción (AND):** usar el índice de menor costo y filtrar el resto en memoria; o índice compuesto; o intersección de identificadores de múltiples índices.
- **Disyunción (OR):** usar índices para todas las condiciones y unir sus resultados; o hacer full scan.

Plan de ejecución

A partir de estadísticas y estimaciones, el optimizer elige un **plan de ejecución** que especifica: qué índices usar, qué algoritmo aplicar en cada operación (selección, join, proyección, etc.). En PostgreSQL se puede inspeccionar con `EXPLAIN` / `EXPLAIN ANALYZE`.

Mapa conceptual del curso

Bloque	Temas y teóricas
Diseño	DER (T2), Modelo Relacional (T3.1), Normalización (T5.2)
Consulta	Álgebra Relacional (T3.2), SQL básico (T3.3), SQL avanzado (T4.1, T4.2), Window Functions (T5.1)
Gestión	Serializabilidad (T6.1), Recuperabilidad (T6.2), Recuperación/UNDO-REDO (T6.3, T7.1), Backups (T7.2)
Implementación	Almacenamiento e Índices (T7.3), Procesamiento de consultas (T7.4)